```
// ==== 源文件: internal/money/money.go ====
// Package money 提供应用内金币的计量与结算原语。
//
// 设计要点：金币全程以 int64 整数存储与运算，任何地方都不出现 float64，
// 以免大额金币（百亿级）在浮点转换时丢精度，也让倍率与分成的结果可复现。
package money
import (
	"errors"
	"strconv"
	"strings"
)
// Coin 表示金币数量，单位为 1 枚金币。
type Coin int64
// ErrOverflow 表示金额运算越过了安全上限。
var ErrOverflow = errors.New("金额超出上限")
// Ceiling 是单笔金额的安全上限，防止异常数据把库里的数值写到越界。
const Ceiling Coin = 1_000_000_000_000_000
// Ratio 用有理数表达倍率，避免 1.5 这类小数进入浮点。
// 例如 1.5 倍记作 {N: 3, D: 2}，2 倍记作 {N: 2, D: 1}。
type Ratio struct {
	N int64
	D int64
}
// NewRatio 构造倍率，分母非法时退化为 1 倍。
func NewRatio(n, d int64) Ratio {
	if d == 0 {
		return Ratio{N: 1, D: 1}
	}
	return Ratio{N: n, D: d}
}
// Valid 判断倍率是否可用。
func (r Ratio) Valid() bool { return r.D > 0 && r.N > 0 }
// Float 仅供界面与日志展示，不参与记账。
func (r Ratio) Float() float64 {
	if r.D == 0 {
		return 1
	}
	return float64(r.N) / float64(r.D)
}
// Text 输出「1 倍」「1.5 倍」这类中文倍率文案。
func (r Ratio) Text() string {
	if !r.Valid() {
		return "1 倍"
	}
	if r.N%r.D == 0 {
		return strconv.FormatInt(r.N/r.D, 10) + "倍"
	}
	whole := r.N / r.D
```

```
	frac := (r.N % r.D) * 10 / r.D
	if frac == 0 {
		return strconv.FormatInt(whole, 10) + "倍"
	}
	return strconv.FormatInt(whole, 10) + "." + strconv.FormatInt(frac, 10) + "倍"
}
// Scale 按倍率放大金额。倍率非法时原样返回，便于调用方把「倍率算错」
// 与「收益为零」区分开。
func (c Coin) Scale(r Ratio) Coin {
	if !r.Valid() {
		return c
	}
	return c.MulDiv(r.N, r.D)
}
// MulDiv 计算 c*n/d，先做乘法再做除法，用一次取整完成比例运算。
func (c Coin) MulDiv(n, d int64) Coin {
	if d == 0 {
		return c
	}
	return Coin(int64(c) * n / d)
}
// PerMinute 把「每小时收益」按已停分钟数折算，不足一分钟的部分不计。
func (c Coin) PerMinute(minutes int) Coin {
	if minutes <= 0 {
		return 0
	}
	return c.MulDiv(int64(minutes), 60)
}
// Split 按千分比把金额切成两份，余数归第一位。
// 保证 a+b 恰好等于原金额，不会出现四舍五入后总额相差一枚金币的情况。
func (c Coin) Split(permille int64) (Coin, Coin) {
	if permille < 0 {
		permille = 0
	}
	if permille > 1000 {
		permille = 1000
	}
	first := c.MulDiv(permille, 1000)
	return first, c - first
}
// Negative 返回相反数，用于扣款表达式。
func (c Coin) Negative() Coin { return -c }
// Positive 保证金额不为负，负值一律归零后再入账。
func (c Coin) Positive() Coin {
	if c < 0 {
		return 0
	}
	return c
```

```
}
// String 输出带千分位的金额文案，供接口与日得日志使用。
func (c Coin) String() string {
	neg := c < 0
	digits := strconv.FormatInt(int64(c), 10)
	if neg {
		digits = digits[1:]
	}
	var b strings.Builder
	for i, ch := range digits {
		if i > 0 && (len(digits)-i)%3 == 0 {
			b.WriteByte(',')
		}
		b.WriteRune(ch)
	}
	if neg {
		return "-" + b.String()
	}
	return b.String()
}
// Parse 解析玩家输入或配置文件里的金额文本，允许下划线与千分位逗号。
func Parse(text string) (Coin, error) {
	clean := strings.NewReplacer(",", "", "_", "", " ", "").Replace(text)
	if clean == "" {
		return 0, errors.New("金额为空")
	}
	v, err := strconv.ParseInt(clean, 10, 64)
	if err != nil {
		return 0, err
	}
	return Coin(v), nil
}
// ==== 源文件: internal/gameplay/stage.go ====
package gameplay
import (
	"errors"
	"time"
)
// Stage 描述一辆车停进车位后所处的时间阶段。
// 阶段是判定操作权限的唯一依据，不依赖调用方自己算分钟数。
type Stage uint8
const (
	// StageLock 绝对锁定期：车主不能召回，车位主人不能贴条。
	StageLock Stage = iota + 1
	// StageEscape 逃生期：车主可召回，车位主人仍不能贴条。
	StageEscape
	// StageHunt 争夺期：车主可召回，车位主人可贴条。
	StageHunt
```

```
	// StageMature 期满：收益走满额结算，不再受操作影响。
	StageMature
)
// Code 返回用于接口与素材的英文短码。
func (s Stage) Code() string {
	switch s {
	case StageLock:
		return "lock"
	case StageEscape:
		return "escape"
	case StageHunt:
		return "hunt"
	case StageMature:
		return "mature"
	}
	return "unknown"
}
// Label 返回阶段名称，界面直接展示。
func (s Stage) Label() string {
	switch s {
	case StageLock:
		return "绝对锁定期"
	case StageEscape:
		return "逃生观察期"
	case StageHunt:
		return "争夺期"
	case StageMature:
		return "期满"
	}
	return "未知阶段"
}
// Hint 返回面向玩家的阶段说明，用在卡片浮层里。
func (s Stage) Hint() string {
	switch s {
	case StageLock:
		return "刚停进来，双方都不能动它"
	case StageEscape:
		return "车主可以召回了，车位主人还贴不了条"
	case StageHunt:
		return "车位主人可以贴条了，谁先上手谁占便宜"
	case StageMature:
		return "已经停满，收益按满额算"
	}
	return ""
}
// Act 表示一次针对车辆或车位的操作。
type Act uint8
const (
```

```
	// ActPark 把空闲车辆派出去。
	ActPark Act = 1 << iota
	// ActRecall 车主主动召回。
	ActRecall
	// ActTicket 车位主人贴条。
	ActTicket
	// ActCard 车主用幸运卡即时结算。
	ActCard
	// ActSell 车主把车卖掉。
	ActSell
)
// Code 返回操作的英文短码。
func (a Act) Code() string {
	switch a {
	case ActPark:
		return "park"
	case ActRecall:
		return "recall"
	case ActTicket:
		return "ticket"
	case ActCard:
		return "card"
	case ActSell:
		return "sell"
	}
	return "unknown"
}
// ActSet 是操作集合的位图，用来表达「某阶段允许哪些操作」。
type ActSet uint16
// Has 判断集合里是否含有某个操作。
func (s ActSet) Has(a Act) bool { return uint16(s)&uint16(a) != 0 }
// List 以短码数组形式导出集合，便于前端按需渲染按钮。
func (s ActSet) List() []string {
	out := make([]string, 0, 5)
	for _, a := range []Act{ActPark, ActRecall, ActTicket, ActCard, ActSell} {
		if s.Has(a) {
			out = append(out, a.Code())
		}
	}
	return out
}
// stageActs 是「阶段 → 允许操作」的权限表。
// 把规则集中在这一张表里，规则调整只改这里，服务层不需要跟着改判断。
var stageActs = map[Stage]ActSet{
	StageLock:   ActSet(ActCard),
	StageEscape: ActSet(ActRecall | ActCard),
	StageHunt:   ActSet(ActRecall | ActCard | ActTicket),
	StageMature: ActSet(ActRecall | ActTicket),
```

```
}
// Allowed 查询某阶段是否允许某操作。
func Allowed(s Stage, a Act) bool {
	set, ok := stageActs[s]
	if !ok {
		return false
	}
	return set.Has(a)
}
// Acts 返回某阶段允许的全部操作。
func Acts(s Stage) ActSet {
	if set, ok := stageActs[s]; ok {
		return set
	}
	return 0
}
// Timeline 描述一段租约的时间轴分段。
// 默认是 1 小时锁定 + 1 小时观察 + 6 小时争夺，期满共 8 小时。
type Timeline struct {
	LockFor   time.Duration
	EscapeFor time.Duration
	HuntFor   time.Duration
}
// DefaultTimeline 返回标准时间轴。
func DefaultTimeline() Timeline {
	return Timeline{
		LockFor:   time.Hour,
		EscapeFor: time.Hour,
		HuntFor:   6 * time.Hour,
	}
}
// Total 返回从停入到自然期满的总时长。
func (t Timeline) Total() time.Duration {
	return t.LockFor + t.EscapeFor + t.HuntFor
}
// Validate 检查时间轴是否单调递增，配置写错时启动即报错。
func (t Timeline) Validate() error {
	if t.LockFor <= 0 || t.EscapeFor <= 0 || t.HuntFor <= 0 {
		return errors.New("时间轴各段必须大于零")
	}
	if t.Total() > 48*time.Hour {
		return errors.New("时间轴总长超过 48 小时，请检查配置")
	}
	return nil
}
// Elapsed 返回已经停放的实际时长，起始时间在未来时按零处理。
func (t Timeline) Elapsed(start, now time.Time) time.Duration {
	if start.After(now) {
```

```
		return 0
	}
	return now.Sub(start)
}
// StageOf 按起点与当前时刻判定阶段。
func (t Timeline) StageOf(start, now time.Time) Stage {
	spent := t.Elapsed(start, now)
	switch {
	case spent < t.LockFor:
		return StageLock
	case spent < t.LockFor+t.EscapeFor:
		return StageEscape
	case spent < t.Total():
		return StageHunt
	default:
		return StageMature
	}
}
// Remaining 返回距离下一阶段还差多久，期满后返回零。
func (t Timeline) Remaining(start, now time.Time) time.Duration {
	spent := t.Elapsed(start, now)
	if spent >= t.Total() {
		return 0
	}
	next := t.LockFor
	if spent >= t.LockFor {
		next = t.LockFor + t.EscapeFor
	}
	if spent >= t.LockFor+t.EscapeFor {
		next = t.Total()
	}
	return next - spent
}
// VehicleState 是车辆在租约中的两种状态。
type VehicleState uint8
const (
	// VehicleIdle 车辆停在自家车库。
	VehicleIdle VehicleState = 0
	// VehicleLeased 车辆已被派往他人车位。
	VehicleLeased VehicleState = 1
)
// CellState 是车位的两种状态。
type CellState uint8
const (
	// CellFree 车位空着。
	CellFree CellState = 0
	// CellTaken 车位已被占用。
	CellTaken CellState = 1
```

```
)
// ==== 源文件: internal/gameplay/quote.go ====
package gameplay
import "tstc/internal/money"
// Quote 是一次停车能拿到的倍率报价。
// Gap 为停放方等级减车位主人等级，正数是「向下兼容」，负数是「向上攀附」。
type Quote struct {
	Gap   int
	Ratio money.Ratio
}
// 倍率阶梯常量：这些数字同时被说明书与数值表引用，抽出来避免散落。
const (
	permilleDown   = 1000 // 1 倍
	permillePeer   = 1500 // 1.5 倍
	permilleUpward = 2000 // 2 倍
)
// QuoteFor 按双方等级差给出倍率。
// 停放方等级不低于车位主人时按 1 倍，平级 1.5 倍，低一级及以上 2 倍。
func QuoteFor(parkerLevel, hostLevel int) Quote {
	gap := parkerLevel - hostLevel
	switch {
	case gap > 0:
		return Quote{Gap: gap, Ratio: money.NewRatio(permilleDown, 1000)}
	case gap == 0:
		return Quote{Gap: 0, Ratio: money.NewRatio(permillePeer, 1000)}
	default:
		return Quote{Gap: gap, Ratio: money.NewRatio(permilleUpward, 1000)}
	}
}
// Text 返回「2 倍」这类展示文案。
func (q Quote) Text() string { return q.Ratio.Text() }
// Note 返回倍率来由的说明，用于停车成功后的提示。
func (q Quote) Note() string {
	switch {
	case q.Gap > 0:
		return "对方等级比你低，按 1 倍结算"
	case q.Gap == 0:
		return "等级相同，按 1.5 倍结算"
	default:
		return "对方等级比你高，按 2 倍结算"
	}
}
// ==== 源文件: internal/gameplay/settle.go ====
package gameplay
import (
	"errors"
	"time"
	"tstc/internal/money"
```

```
)
// Kind 表示一笔租约以什么方式收尾。
type Kind uint8
const (
	// KindNone 表示租约还在进行中。
	KindNone Kind = iota
	// KindRecall 车主在可召回阶段主动开走。
	KindRecall
	// KindTicket 车位主人贴条把车赶走。
	KindTicket
	// KindMature 停满时长自然期满。
	KindMature
	// KindCard 车主使用幸运卡即时结算。
	KindCard
)
// Code 返回结算方式的英文短码。
func (k Kind) Code() string {
	switch k {
	case KindRecall:
		return "recall"
	case KindTicket:
		return "ticket"
	case KindMature:
		return "mature"
	case KindCard:
		return "card"
	}
	return "none"
}
// Label 返回结算方式的中文名。
func (k Kind) Label() string {
	switch k {
	case KindRecall:
		return "主动召回"
	case KindTicket:
		return "贴条结算"
	case KindMature:
		return "期满结算"
	case KindCard:
		return "幸运卡秒结"
	}
	return ""
}
// Who 表示发起操作的一方，用于校验操作资格。
type Who uint8
const (
	// WhoParker 是停放车辆的一方。
	WhoParker Who = iota + 1
```

```
	// WhoHost 是车位的所有权人。
	WhoHost
)
// 结算环节可能返回的错误，接口层统一映射成提示文案。
var (
	// ErrWrongSide 表示操作方没有这项权限。
	ErrWrongSide = errors.New("你没有这项操作权限")
	// ErrPhaseClosed 表示当前阶段不允许该操作。
	ErrPhaseClosed = errors.New("当前阶段不允许该操作")
	// ErrBadAmount 表示金额异常（负数或越过上限）。
	ErrBadAmount = errors.New("收益金额异常")
)
// Policy 收纳一切可调的结算比例。
// 比例用千分数表达：900 即 90%，避免 0.9 之类的浮点进入账目。
type Policy struct {
	RecallPermille      int64
	TicketParkerPermille int64
	FullHours           int
}
// DefaultPolicy 返回出厂结算策略：召回 90%，贴条 80/20，满 8 小时。
func DefaultPolicy() Policy {
	return Policy{
		RecallPermille:      900,
		TicketParkerPermille: 800,
		FullHours:           8,
	}
}
// Validate 检查策略是否落在合理区间，配置错误时启动即失败。
func (p Policy) Validate() error {
	if p.FullHours <= 0 || p.FullHours > 24 {
		return errors.New("满额时长必须在 1 到 24 小时之间")
	}
	if p.RecallPermille <= 0 || p.RecallPermille > 1000 {
		return errors.New("召回比例必须在 0 到 1000 千分之间")
	}
	if p.TicketParkerPermille <= 0 || p.TicketParkerPermille >= 1000 {
		return errors.New("贴条分成比例必须在 0 到 1000 千分之间")
	}
	return nil
}
// Slip 是一张结算单，把一次结算的全部数值固化下来，便于入库与回溯。
type Slip struct {
	Kind      Kind
	SettledAt time.Time
	Minutes   int
	Hourly    money.Coin
	Rate      money.Ratio
	Gross     money.Coin
```

```
    ParkerShare money.Coin
    HostShare   money.Coin
    Reason      string
}
// Total 返回这张单子实际发出去的金币总数。
func (s Slip) Total() money.Coin { return s.ParkerShare + s.HostShare }
// Adjudicate 依据结算方式、时间轴与倍率生成结算单。
//
// 调用方只需给出「谁发起、什么方式、起点在哪」，权限判定与金额计算都在这里完成，
// 上层不需要重复写 if 判断阶段。
func (p Policy) Adjudicate(kind Kind, by Who, start, now time.Time,
    hourly money.Coin, quote Quote, tl Timeline) (Slip, error) {
    if hourly < 0 {
        return Slip{}, ErrBadAmount
    }
    slip := Slip{
        Kind:      kind,
        SettledAt: now,
        Hourly:    hourly,
        Rate:      quote.Ratio,
        Minutes:   int(tl.Elapsed(start, now).Minutes()),
    }
    if slip.Minutes < 0 {
        slip.Minutes = 0
    }
    stage := tl.StageOf(start, now)
    switch kind {
    case KindCard:
        if by != WhoParker {
            return Slip{}, ErrWrongSide
        }
        // 幸运卡随时可用，直接按满额计算，不受当前停留时长影响。
        slip.Gross = hourly.Scale(quote.Ratio).MulDiv(int64(p.FullHours), 1)
        slip.ParkerShare = slip.Gross
        slip.Reason = "幸运卡按满额即时结算"
    case KindRecall:
        if by != WhoParker {
            return Slip{}, ErrWrongSide
        }
        if !Allowed(stage, ActRecall) {
            return Slip{}, ErrPhaseClosed
        }
        slip.Gross = hourly.Scale(quote.Ratio).PerMinute(slip.Minutes)
        slip.ParkerShare, slip.HostShare = slip.Gross.Split(p.RecallPermille)
        slip.HostShare = 0
        slip.Reason = "车主主动召回，扣除部分收益"
    case KindTicket:
        if by != WhoHost {
```

```
                return Slip{}, ErrWrongSide
            }
            if !Allowed(stage, ActTicket) {
                return Slip{}, ErrPhaseClosed
            }
            slip.Gross = hourly.Scale(quote.Ratio).PerMinute(slip.Minutes)
            slip.ParkerShare, slip.HostShare = slip.Gross.Split(p.TicketParkerPermille)
            slip.Reason = "车位主人贴条，按比例分成"
        case KindMature:
            // 期满结算不需要任何人发起，只要阶段到了期满就直接成交。
            if stage != StageMature {
                return Slip{}, ErrPhaseClosed
            }
            slip.Gross = hourly.Scale(quote.Ratio).MulDiv(int64(p.FullHours), 1)
            slip.ParkerShare = slip.Gross
            slip.Reason = "停放期满，收益全额归车主"
        default:
            return Slip{}, ErrPhaseClosed
        }
        if slip.ParkerShare < 0 || slip.HostShare < 0 {
            return Slip{}, ErrBadAmount
        }
        if slip.Total() > money.Ceiling {
            return Slip{}, ErrBadAmount
        }
        return slip, nil
}
// Summary 生成一句面向玩家的结算播报。
func (s Slip) Summary() string {
        switch s.Kind {
        case KindRecall:
            return "召回成功，按 " + s.Rate.Text() + " 结算"
        case KindTicket:
            return "车位被贴条，按 " + s.Rate.Text() + " 分账"
        case KindMature:
            return "停满 " + s.Rate.Text() + " 的满额收益已入账"
        case KindCard:
            return "幸运卡秒结，按 " + s.Rate.Text() + " 拿满额"
        }
        return ""
}
// ==== 源文件: internal/gameplay/catalog.go ====
package gameplay
import (
        "math"
        "sync"
        "tstc/internal/money"
)
```

```
// Vehicle 是一档车型的图鉴条目。
type Vehicle struct {
	Level  int
	Name   string
	Price  money.Coin
	Hourly money.Coin
}
// FullIncome 返回该车型在指定倍率下停满满额时长的收入。
func (v Vehicle) FullIncome(p Policy, r money.Ratio) money.Coin {
	return v.Hourly.Scale(r).MulDiv(int64(p.FullHours), 1)
}
// TradeIn 返回按回收比例折算后的卖车价。
func (v Vehicle) TradeIn(permille int64) money.Coin {
	return v.Price.MulDiv(permille, 1000)
}
// anchor 是图鉴的锚点数据：只写关键等级，中间等级由生成器插值补齐。
type anchor struct {
	Level  int
	Name   string
	Price  int64
	Hourly int64
}
// anchors 对应数值表里的关键档位，价格指数增长、产出线性增长。
var anchors = []anchor{
	{1, "破旧拖拉机", 100, 100},
	{2, "二八大杠自行车", 500, 150},
	{3, "破烂三轮车", 1_500, 250},
	{4, "报废奥拓", 4_000, 400},
	{5, "宏光面包车", 10_000, 600},
	{6, "经济型代步车", 25_000, 850},
	{7, "城市紧凑 SUV", 60_000, 1_150},
	{8, "雅阁商务车", 120_000, 1_500},
	{9, "猛禽皮卡", 250_000, 2_000},
	{10, "宝马 3 系轿跑", 500_000, 2_800},
	{20, "保时捷 718", 4_000_000, 8_000},
	{30, "迈巴赫 S 级", 20_000_000, 18_000},
	{40, "玛莎拉蒂总裁", 80_000_000, 35_000},
	{50, "劳斯莱斯幻影", 300_000_000, 60_000},
	{60, "蝙蝠车", 1_000_000_000, 100_000},
	{70, "纯金重型坦克", 4_000_000_000, 180_000},
	{80, "阿帕奇武装直升机", 15_000_000_000, 300_000},
	{90, "歼十战斗机", 50_000_000_000, 500_000},
	{100, "赛博悬浮 UFO", 150_000_000_000, 800_000},
}
// filler 是插值生成等级时使用的中性名称前缀，避免大量重复的动物名。
var filler = []string{"家用型", "改装型", "竞速型", "越野型", "收藏级", "限定版", "实验型", "概念型", "典藏版", "传奇版"}
var (
	catalogOnce sync.Once
```

```
	catalogList []Vehicle
)
// Catalog 返回 1 到 100 级的完整图鉴，首次调用时生成并缓存。
func Catalog() []Vehicle {
	catalogOnce.Do(func() {
		catalogList = buildCatalog()
	})
	return catalogList
}
// VehicleAt 按等级取图鉴条目，等级越界时返回 false。
func VehicleAt(level int) (Vehicle, bool) {
	if level < 1 || level > 100 {
		return Vehicle{}, false
	}
	list := Catalog()
	return list[level-1], true
}
// buildCatalog 把锚点之间的空档用等比插值补齐。
//
// 价格在两级锚点之间按等比数列递增（复利式膨胀），产出按等差数列递增，
// 这样既能保留数值表的关键档位，又能让每一级都有可购买的具体车型。
func buildCatalog() []Vehicle {
	out := make([]Vehicle, 0, 100)
	for i := 0; i < len(anchors)-1; i++ {
		from, to := anchors[i], anchors[i+1]
		gap := to.Level - from.Level
		priceStep := math.Pow(float64(to.Price)/float64(from.Price), 1/float64(gap))
		hourlyStep := float64(to.Hourly-from.Hourly) / float64(gap)
		for step := 0; step < gap; step++ {
			level := from.Level + step
			name := from.Name
			if step > 0 {
				name = filler[(level+step)%len(filler)] + from.Name
			}
			price := float64(from.Price) * math.Pow(priceStep, float64(step))
			hourly := float64(from.Hourly) + hourlyStep*float64(step)
			out = append(out, Vehicle{
				Level:  level,
				Name:   name,
				Price:  money.Coin(roundTo(price, 10)),
				Hourly: money.Coin(roundTo(hourly, 10)),
			})
		}
	}
	last := anchors[len(anchors)-1]
	out = append(out, Vehicle{
		Level:  last.Level,
		Name:   last.Name,
```

```
            Price:    money.Coin(last.Price),
            Hourly: money.Coin(last.Hourly),
        })
        return out
}
// roundTo 把浮点结果对齐到给定的步长，让图鉴价格读起来是整数。
func roundTo(v float64, step float64) float64 {
        if step <= 0 {
            return math.Round(v)
        }
        return math.Round(v/step) * step
}
// ResalePermille 是卖车回收比例（千分数）。
const ResalePermille int64 = 850
// UnlockMaxLevel 返回玩家当前可购买的车型等级上限。
func UnlockMaxLevel(playerLevel int) int {
        if playerLevel < 1 {
            return 1
        }
        if playerLevel > 100 {
            return 100
        }
        return playerLevel
}
// ==== 源文件: internal/gameplay/garage.go ====
package gameplay
import (
        "hash/fnv"
        "strings"
        "tstc/internal/money"
)
// 车库与车位的容量规则：随等级增长，最终都封顶 10 个。
const (
        CapCeiling          = 10
        lotsBase            = 1
        lotsPerLevels     = 11
        garageBase          = 3
        garagePerLevels = 9
)
// Capacity 描述一个玩家的车库与车位额度。
type Capacity struct {
        VehicleSlots int
        LotCells         int
}
// CapacityFor 按等级算额度：车位每 11 级加一个，车库每 9 级加一个，均不超过 10。
func CapacityFor(level int) Capacity {
        if level < 1 {
            level = 1
```

```
	}
	cells := lotsBase + level/lotsPerLevels
	if cells > CapCeiling {
		cells = CapCeiling
	}
	slots := garageBase + level/garagePerLevels
	if slots > CapCeiling {
		slots = CapCeiling
	}
	return Capacity{VehicleSlots: slots, LotCells: cells}
}
// 签到档位：普通签到给保底金币，幸运卡签到给豪华金币并附带经验。
const (
	PlainSignCoin  money.Coin = 5_000
	CardSignCoin   money.Coin = 50_000
	CardSignExp                = 20
	CardSignCost               = 1
)
// DailyCardQuota 是每日通过观看激励内容可获得的幸运卡上限。
const DailyCardQuota = 10
// SignWay 表示签到方式。
type SignWay string
const (
	// SignPlain 是免费签到。
	SignPlain SignWay = "plain"
	// SignCard 是消耗一张幸运卡的豪华签到。
	SignCard SignWay = "card"
)
// ParseSignWay 把外部传入的字符串转成签到方式，未知值按非法处理。
func ParseSignWay(raw string) (SignWay, bool) {
	switch strings.ToLower(strings.TrimSpace(raw)) {
	case "plain", "normal", "free":
		return SignPlain, true
	case "card", "lucky":
		return SignCard, true
	}
	return "", false
}
// SignReward 是一次签到的产出。
type SignReward struct {
	Way  SignWay
	Coin money.Coin
	Exp  int
	Cost int
}
// RewardOf 返回该签到方式对应的奖励，供服务层直接入账。
func RewardOf(way SignWay) SignReward {
	if way == SignCard {
```

```
            return SignReward{Way: SignCard, Coin: CardSignCoin, Exp: CardSignExp, Cost: CardSignCost}
        }
        return SignReward{Way: SignPlain, Coin: PlainSignCoin}
}
// LevelUpExp 返回从当前等级升到下一级所需经验（1 级到 100 级）。
// 采用平方根式增长：前期升级快，后期靠高频停车累积。
func LevelUpExp(level int) int {
        if level < 1 {
            level = 1
        }
        if level >= 100 {
            return 0
        }
        base := 120 + level*level*3
        return base
}
// SettleExp 返回一次结算能获得的经验。
// 经验只和停放时长、倍率有关，与金币数额脱钩，避免高等级车型刷经验。
func SettleExp(minutes int, quote Quote) int {
        if minutes <= 0 {
            return 0
        }
        raw := int(int64(minutes) * quote.Ratio.N / quote.Ratio.D / 2)
        if raw > 240 {
            return 240
        }
        return raw
}
// PlayerUID 生成玩家对外展示的短号：五位大写字母加数字，避开易混字符。
func PlayerUID(seed int64) string {
        const alphabet = "23456789ABCDEFGHJKLMNPQRSTUVWXYZ"
        h := fnv.New64a()
        var buf [8]byte
        for i := 0; i < 8; i++ {
            buf[i] = byte(seed >> (8 * i))
        }
        _, _ = h.Write(buf[:])
        sum := h.Sum64()
        out := make([]byte, 5)
        for i := 0; i < 5; i++ {
            out[i] = alphabet[sum%uint64(len(alphabet))]
            sum /= uint64(len(alphabet))
        }
        return string(out)
}
// ==== 源文件: internal/service/service.go ====
// Package service 承载业务编排：校验权限、开事务、调用领域规则、落账。
// 领域规则留在 gameplay，SQL 留在 repo，这里只负责把它们串成一次完整的操作。
```

```
package service
import (
	"errors"
	"strings"
	"time"
	"tstc/internal/gameplay"
	"tstc/internal/money"
	"tstc/internal/repo"
)
// 服务层对外暴露的错误，接口层按这些错误给出中文提示与状态码。
var (
	// ErrBadInput 表示请求参数不合法。
	ErrBadInput = errors.New("请求参数不合法")
	// ErrNoPlayer 表示账号不存在。
	ErrNoPlayer = errors.New("账号不存在")
	// ErrNoCoin 表示余额不足。
	ErrNoCoin = errors.New("金币不足")
	// ErrNoCard 表示幸运卡不足。
	ErrNoCard = errors.New("幸运卡不足")
	// ErrNoFreeLot 表示全服暂时没有空车位。
	ErrNoFreeLot = errors.New("当前没有可用的空车位")
	// ErrSlotFull 表示额度已满。
	ErrSlotFull = errors.New("额度已满")
	// ErrOccupied 表示车位已被抢占。
	ErrOccupied = errors.New("手慢了，车位已被别人占走")
	// ErrNotYours 表示操作了不属于自己的资产。
	ErrNotYours = errors.New("这不是你的资产")
	// ErrSelf 表示把自己当成了操作对象。
	ErrSelf = errors.New("不能对自己做这个操作")
	// ErrLevelLocked 表示等级不足，车型还没解锁。
	ErrLevelLocked = errors.New("该车型尚未解锁")
	// ErrSigned 表示今天已经签到过。
	ErrSigned = errors.New("今天已经签到过了")
	// ErrAdExhausted 表示今日领取次数用完。
	ErrAdExhausted = errors.New("今天的幸运卡已经领完了")
)
// Options 是构造服务所需的全部装配参数。
type Options struct {
	Store    *repo.Store
	Policy   gameplay.Policy
	Timeline gameplay.Timeline
	// Clock 允许测试注入固定时间，生产环境用 time.Now。
	Clock func() time.Time
}
// Service 是业务入口，全部方法都要求传入已登录玩家的 ID。
type Service struct {
	store    *repo.Store
	policy   gameplay.Policy
```

```
	timeline gameplay.Timeline
	clock    func() time.Time
}
// New 构造服务，缺失的连接或非法策略会直接返回错误。
func New(opt Options) (*Service, error) {
	if opt.Store == nil {
		return nil, errors.New("缺少数据源")
	}
	if err := opt.Policy.Validate(); err != nil {
		return nil, err
	}
	if err := opt.Timeline.Validate(); err != nil {
		return nil, err
	}
	clock := opt.Clock
	if clock == nil {
		clock = time.Now
	}
	return &Service{
		store:    opt.Store,
		policy:   opt.Policy,
		timeline: opt.Timeline,
		clock:    clock,
	}, nil
}
// Now 返回服务当前使用的时间。
func (s *Service) Now() time.Time { return s.clock() }
// Today 返回按本地时区格式化的日期，签到与广告额度都以它为界。
func (s *Service) Today() string { return s.clock().Format("2006-01-02") }
// normalizeNick 去掉首尾空白并把连续空白压成一个空格。
func normalizeNick(raw string) string {
	fields := strings.Fields(raw)
	return strings.Join(fields, " ")
}
// validNick 校验昵称长度，过短或过长都退回默认昵称。
func validNick(nick string) bool {
	n := len([]rune(nick))
	return n >= 2 && n <= 12
}
// StageView 是阶段信息在接口里的表达。
type StageView struct {
	Code      string `json:"code"`
	Label     string `json:"label"`
	Hint      string `json:"hint"`
	Minutes   int    `json:"minutes"`
	Remaining int    `json:"remaining"`
}
// stageView 把阶段、已停分钟数、距下一阶段的剩余分钟数打包。
```

```
func (s *Service) stageView(start time.Time) StageView {
	now := s.clock()
	stage := s.timeline.StageOf(start, now)
	return StageView{
		Code:      stage.Code(),
		Label:     stage.Label(),
		Hint:      stage.Hint(),
		Minutes:   int(s.timeline.Elapsed(start, now).Minutes()),
		Remaining: int(s.timeline.Remaining(start, now).Minutes()),
	}
}
// accrue 计算一段租约到目前为止的累计毛收益（未做分成）。
func (s *Service) accrue(start time.Time, hourly money.Coin, quote gameplay.Quote) money.Coin {
	minutes := int(s.timeline.Elapsed(start, s.clock()).Minutes())
	return hourly.Scale(quote.Ratio).PerMinute(minutes)
}
// clampLimit 把分页参数收进合理区间。
func clampLimit(n, def, max int) int {
	if n <= 0 {
		return def
	}
	if n > max {
		return max
	}
	return n
}
// trimUpper 统一玩家短号的写法：去空白并转大写。
// 短号在库里以大写保存，用户输入小写也能找到人。
func trimUpper(raw string) string {
	return strings.ToUpper(strings.TrimSpace(raw))
}
// ==== 源文件: internal/service/account.go ====
package service
import (
	"context"
	"database/sql"
	"errors"
	"tstc/internal/gameplay"
	"tstc/internal/repo"
)
// mapRepoErr 把仓储层错误翻成服务层语义，未识别的错误原样抛出。
func mapRepoErr(err error) error {
	if err == nil {
		return nil
	}
	if errors.Is(err, repo.ErrNoRow) {
		return ErrNoPlayer
	}
```

```
	return err
}
// Profile 是首页顶栏与个人面板需要的全部数字。
type Profile struct {
	ID          int64  `json:"id"`
	UID         string `json:"uid"`
	Nick        string `json:"nick"`
	Level       int    `json:"level"`
	Exp         int    `json:"exp"`
	ExpNeed     int    `json:"exp_need"`
	Coin        string `json:"coin"`
	CoinRaw     int64  `json:"coin_raw"`
	Card        int    `json:"card"`
	AdLeft      int    `json:"ad_left"`
	VehicleCount int   `json:"vehicle_count"`
	VehicleCap  int    `json:"vehicle_cap"`
	CellFree    int    `json:"cell_free"`
	CellTotal   int    `json:"cell_total"`
	MaxBuyLevel int    `json:"max_buy_level"`
	SignedToday bool   `json:"signed_today"`
	Streak      int    `json:"streak"`
	FullHours   int    `json:"full_hours"`
	RecallPct   int64  `json:"recall_pct"`
	TicketPct   int64  `json:"ticket_pct"`
	CardQuota   int    `json:"card_quota"`
}
// Profile 汇总当前玩家的资产与规则参数，界面只调这一个接口就能画完顶栏。
func (s *Service) Profile(ctx context.Context, playerID int64) (Profile, error) {
	p, err := s.store.PlayerByID(ctx, playerID)
	if err != nil {
		return Profile{}, mapRepoErr(err)
	}
	cells, err := s.store.CellsOf(ctx, playerID)
	if err != nil {
		return Profile{}, err
	}
	vehicles, err := s.store.CountVehicles(ctx, s.store, playerID)
	if err != nil {
		return Profile{}, err
	}
	signed, err := s.store.SignedToday(ctx, playerID, s.Today())
	if err != nil {
		return Profile{}, err
	}
	streak, err := s.store.SignStreak(ctx, playerID, s.Now())
	if err != nil {
		return Profile{}, err
	}
```

```
	limit := p.Capacity()
	free := 0
	for _, c := range cells {
		if c.State == gameplay.CellFree {
			free++
		}
	}
	return Profile{
		ID:           p.ID,
		UID:          p.UID,
		Nick:         p.Nick,
		Level:        p.Level,
		Exp:          p.Exp,
		ExpNeed:      gameplay.LevelUpExp(p.Level),
		Coin:         p.Coin.String(),
		CoinRaw:      int64(p.Coin),
		Card:         p.Card,
		AdLeft:       repo.AdSlot(p, s.Today(), gameplay.DailyCardQuota),
		VehicleCount: vehicles,
		VehicleCap:   limit.VehicleSlots,
		CellFree:     free,
		CellTotal:    len(cells),
		MaxBuyLevel:  gameplay.UnlockMaxLevel(p.Level),
		SignedToday:  signed,
		Streak:       streak,
		FullHours:    s.policy.FullHours,
		RecallPct:    s.policy.RecallPermille / 10,
		TicketPct:    s.policy.TicketParkerPermille / 10,
		CardQuota:    gameplay.DailyCardQuota,
	}, nil
}
// SignResult 是一次签到的回执。
type SignResult struct {
	Way        string `json:"way"`
	Coin       string `json:"coin"`
	Exp        int    `json:"exp"`
	CardSpent int    `json:"card_spent"`
	Level      int    `json:"level"`
	Streak     int    `json:"streak"`
}
// Sign 执行每日签到。
//
// 同一账号同一天只允许一次：先查一次用于给出友好提示，真正的防重由
// day_sign 的唯一键承担，即使两个请求同时到达也只有一条能写进去。
func (s *Service) Sign(ctx context.Context, playerID int64, rawWay string) (SignResult, error) {
	way, ok := gameplay.ParseSignWay(rawWay)
	if !ok {
		return SignResult{}, ErrBadInput
```

```
}
reward := gameplay.RewardOf(way)
day := s.Today()
var out SignResult
err := s.store.InTx(ctx, func(tx *sql.Tx) error {
    p, err := s.store.LockPlayer(ctx, tx, playerID)
    if err != nil {
        return mapRepoErr(err)
    }
    signed, err := s.store.SignedToday(ctx, playerID, day)
    if err != nil {
        return err
    }
    if signed {
        return ErrSigned
    }
    if reward.Cost > 0 && p.Card < reward.Cost {
        return ErrNoCard
    }
    if err := s.store.RecordSign(ctx, tx, playerID, day, way, reward.Coin); err != nil {
        if errors.Is(err, repo.ErrDuplicate) {
            return ErrSigned
        }
        return err
    }
    if reward.Cost > 0 {
        if err := s.store.AddCard(ctx, tx, playerID, -reward.Cost); err != nil {
            return ErrNoCard
        }
    }
    if err := s.store.AddCoin(ctx, tx, playerID, reward.Coin); err != nil {
        return err
    }
    level, _, err := s.store.GainExp(ctx, tx, playerID, reward.Exp)
    if err != nil {
        return err
    }
    streak, err := s.store.SignStreak(ctx, playerID, s.Now())
    if err != nil {
        return err
    }
    out = SignResult{
        Way:       string(way),
        Coin:      reward.Coin.String(),
        Exp:       reward.Exp,
        CardSpent: reward.Cost,
        Level:     level,
        Streak:    streak,
```

```
            }
            return nil
        })
        if err != nil {
            return SignResult{}, err
        }
        return out, nil
}
// CardClaim 是领取幸运卡的回执。
type CardClaim struct {
        Card       int `json:"card"`
        Used       int `json:"used"`
        Left       int `json:"left"`
        Quota      int `json:"quota"`
}
// ClaimCard 处理「看完激励内容领一张幸运卡」。
// 次数按自然日重置，用到的额度记在玩家行的 ad_used / ad_day 两个字段上。
func (s *Service) ClaimCard(ctx context.Context, playerID int64) (CardClaim, error) {
        day := s.Today()
        var out CardClaim
        err := s.store.InTx(ctx, func(tx *sql.Tx) error {
            card, used, err := s.store.ClaimAdCard(ctx, tx, playerID, day, gameplay.DailyCardQuota)
            if err != nil {
                if errors.Is(err, repo.ErrConflict) {
                    return ErrAdExhausted
                }
                return mapRepoErr(err)
            }
            out = CardClaim{
                Card:    card,
                Used:    used,
                Left:    gameplay.DailyCardQuota - used,
                Quota: gameplay.DailyCardQuota,
            }
            return nil
        })
        if err != nil {
            return CardClaim{}, err
        }
        return out, nil
}
// NameCard 是搜索玩家时弹出的名片。
type NameCard struct {
        ID              int64    `json:"id"`
        UID             string `json:"uid"`
        Nick            string `json:"nick"`
        Level           int       `json:"level"`
        CellFree        int       `json:"cell_free"`
```

```
	CellTotal    int       `json:"cell_total"`
	Relation     string `json:"relation"`
	RateIfPark string `json:"rate_if_park"`
}
// SearchPlayer 按短号找人，顺带给出「去他家停能拿几倍」的即时判断。
func (s *Service) SearchPlayer(ctx context.Context, selfID int64, uid string) (NameCard, error) {
	uid = trimUpper(uid)
	if len(uid) != 5 {
		return NameCard{}, ErrBadInput
	}
	target, err := s.store.SearchByUID(ctx, uid)
	if err != nil {
		return NameCard{}, mapRepoErr(err)
	}
	if target.ID == selfID {
		return NameCard{}, ErrSelf
	}
	cells, err := s.store.CellsOf(ctx, target.ID)
	if err != nil {
		return NameCard{}, err
	}
	free := 0
	for _, c := range cells {
		if c.State == gameplay.CellFree {
			free++
		}
	}
	self, err := s.store.PlayerByID(ctx, selfID)
	if err != nil {
		return NameCard{}, mapRepoErr(err)
	}
	relation, err := s.store.RelationKinds(ctx, s.store, selfID, target.ID)
	if err != nil {
		return NameCard{}, err
	}
	return NameCard{
		ID:          target.ID,
		UID:         target.UID,
		Nick:        target.Nick,
		Level:       target.Level,
		CellFree:    free,
		CellTotal:   len(cells),
		Relation:    relation,
		RateIfPark: gameplay.QuoteFor(self.Level, target.Level).Text(),
	}, nil
}
// Rename 修改昵称，长度不合规时退回原昵称并返回参数错误。
func (s *Service) Rename(ctx context.Context, playerID int64, nick string) error {
```

```
	clean := normalizeNick(nick)
	if !validNick(clean) {
		return ErrBadInput
	}
	return s.store.BumpNick(ctx, playerID, clean)
}
// ==== 源文件: internal/service/register.go ====
package service
import (
	"context"
	"database/sql"
	"tstc/internal/gameplay"
	"tstc/internal/money"
	"tstc/internal/repo"
)
// 新手开局资源：一辆免费的车、一点启动金币、一张幸运卡。
const (
	starterCoin      money.Coin = 8_000
	starterCard                 = 1
	starterLevel                = 1
	starterVehicleLevel         = 1
)
// RegisterResult 是建号回执，客户端拿到后直接进首页。
type RegisterResult struct {
	Player   Profile `json:"player"`
	Starter string   `json:"starter"`
}
// Register 建立一个新账号。
//
// 建号是一个原子操作：玩家行、新手车、车位格子必须同时成功，
// 否则会出现「有账号没车」或「有车没车位」的残缺数据。
func (s *Service) Register(ctx context.Context, nick string, seed int64) (RegisterResult, error) {
	name := normalizeNick(nick)
	if !validNick(name) {
		name = "新来的车主"
	}
	uid := gameplay.PlayerUID(seed)
	candidate := repo.Player{
		UID:      uid,
		Nick:     name,
		Avatar: "",
		Level:   1,
		Exp:      0,
		Coin:     starterCoin,
		Card:     starterCard,
		AdDay:   s.Today(),
	}
	var playerID int64
```

```
	err := s.store.InTx(ctx, func(tx *sql.Tx) error {
		id, err := s.store.InsertPlayer(ctx, tx, candidate)
		if err != nil {
			return err
		}
		playerID = id
		if _, err := s.store.InsertVehicle(ctx, tx, id, starterVehicleLevel); err != nil {
			return err
		}
		cells := gameplay.CapacityFor(starterLevel).LotCells
		return s.store.EnsureCells(ctx, tx, id, cells)
	})
	if err != nil {
		return RegisterResult{}, err
	}
	profile, err := s.Profile(ctx, playerID)
	if err != nil {
		return RegisterResult{}, err
	}
	return RegisterResult{
		Player:   profile,
		Starter: "开局赠送一辆破旧拖拉机与 8,000 金币",
	}, nil
}
// EnsureCells 按当前等级补齐车位，登录时调用一次即可。
func (s *Service) EnsureCells(ctx context.Context, playerID int64) error {
	p, err := s.store.PlayerByID(ctx, playerID)
	if err != nil {
		return mapRepoErr(err)
	}
	return s.store.EnsureCells(ctx, s.store, playerID, p.Capacity().LotCells)
}
// ==== 源文件: internal/service/garage.go ====
package service
import (
	"context"
	"database/sql"
	"time"
	"tstc/internal/gameplay"
	"tstc/internal/money"
	"tstc/internal/repo"
)
// ParkView 是「这辆车现在停在哪、还能拿多少」的完整描述。
type ParkView struct {
	HostID    int64     `json:"host_id"`
	HostNick  string    `json:"host_nick"`
	HostLevel int       `json:"host_level"`
	Rate      string    `json:"rate"`
```

```
	Note		string		`json:"note"`
	Stage		StageView	`json:"stage"`
	Accrued		string		`json:"accrued"`
	RecallPay	string		`json:"recall_pay"`
	CardPay		string		`json:"card_pay"`
	CanRecall	bool		`json:"can_recall"`
	CanCard		bool		`json:"can_card"`
}
// VehicleItem 是车库列表里的一项。
type VehicleItem struct {
	ID		int64		`json:"id"`
	Level		int		`json:"level"`
	Name		string		`json:"name"`
	Hourly		string		`json:"hourly"`
	Price		string		`json:"price"`
	TradeIn		string		`json:"trade_in"`
	State		string		`json:"state"`
	Parked		*ParkView	`json:"parked,omitempty"`
	Actions		[]string	`json:"actions"`
}
// Garage 返回玩家车库的全部车辆，停在外面的车会带上租约信息。
func (s *Service) Garage(ctx context.Context, playerID int64) ([]VehicleItem, error) {
	self, err := s.store.PlayerByID(ctx, playerID)
	if err != nil {
		return nil, mapRepoErr(err)
	}
	cards, err := s.store.VehiclesWithLease(ctx, playerID)
	if err != nil {
		return nil, err
	}
	out := make([]VehicleItem, 0, len(cards))
	for _, c := range cards {
		spec, ok := gameplay.VehicleAt(c.Level)
		if !ok {
			continue
		}
		item := VehicleItem{
			ID:		c.ID,
			Level:		c.Level,
			Name:		spec.Name,
			Hourly:		spec.Hourly.String(),
			Price:		spec.Price.String(),
			TradeIn:	spec.TradeIn(gameplay.ResalePermille).String(),
			State:		"idle",
			Actions:	[]string{gameplay.ActPark.Code(), gameplay.ActSell.Code()},
		}
		if c.State == gameplay.VehicleLeased && c.BeganAt != nil {
			item.State = "parked"
```

```
                view := s.parkView(*c.BeganAt, spec.Hourly, self.Level, c.HostLevel, self.Card)
                view.HostID = c.HostID
                view.HostNick = c.HostNick
                item.Parked = &view
                item.Actions = []string{}
                if view.CanRecall {
                        item.Actions = append(item.Actions, gameplay.ActRecall.Code())
                }
                if view.CanCard {
                        item.Actions = append(item.Actions, gameplay.ActCard.Code())
                }
        }
        out = append(out, item)
    }
    return out, nil
}
// parkView 计算一辆停车中的车在界面上的全部派生字段。
func (s *Service) parkView(start time.Time, hourly money.Coin,
    parkerLv, hostLv, cardLeft int) ParkView {
    quote := gameplay.QuoteFor(parkerLv, hostLv)
    view := ParkView{
        Rate:      quote.Text(),
        Note:      quote.Note(),
        Stage:     s.stageView(start),
        Accrued:   s.accrue(start, hourly, quote).String(),
        CanCard:   cardLeft > 0,
        CanRecall: gameplay.Allowed(s.timeline.StageOf(start, s.clock()), gameplay.ActRecall),
    }
    if slip, err := s.policy.Adjudicate(gameplay.KindRecall, gameplay.WhoParker,
        start, s.clock(), hourly, quote, s.timeline); err == nil {
        view.RecallPay = slip.ParkerShare.String()
    } else {
        view.RecallPay = "0"
    }
    if slip, err := s.policy.Adjudicate(gameplay.KindCard, gameplay.WhoParker,
        start, s.clock(), hourly, quote, s.timeline); err == nil {
        view.CardPay = slip.ParkerShare.String()
    }
    return view
}
// TradeResult 是买车与卖车的共同回执。
type TradeResult struct {
    Action string `json:"action"`
    Level  int    `json:"level"`
    Name   string `json:"name"`
    Coin   string `json:"coin"`
    Delta  string `json:"delta"`
}
```

```
// Buy 购买一辆指定等级的车辆。
// 校验顺序刻意按「等级解锁 → 额度 → 余额」排，让玩家先看到最该解决的问题。
func (s *Service) Buy(ctx context.Context, playerID int64, level int) (TradeResult, error) {
	spec, ok := gameplay.VehicleAt(level)
	if !ok {
		return TradeResult{}, ErrBadInput
	}
	p, err := s.store.PlayerByID(ctx, playerID)
	if err != nil {
		return TradeResult{}, mapRepoErr(err)
	}
	if level > gameplay.UnlockMaxLevel(p.Level) {
		return TradeResult{}, ErrLevelLocked
	}
	if p.Coin < spec.Price {
		return TradeResult{}, ErrNoCoin
	}
	count, err := s.store.CountVehicles(ctx, s.store, playerID)
	if err != nil {
		return TradeResult{}, err
	}
	if count >= p.Capacity().VehicleSlots {
		return TradeResult{}, ErrSlotFull
	}
	err = s.store.InTx(ctx, func(tx *sql.Tx) error {
		if err := s.store.AddCoin(ctx, tx, playerID, spec.Price.Negative()); err != nil {
			return ErrNoCoin
		}
		if _, err := s.store.InsertVehicle(ctx, tx, playerID, level); err != nil {
			return err
		}
		return nil
	})
	if err != nil {
		return TradeResult{}, err
	}
	return TradeResult{
		Action: "buy",
		Level:  level,
		Name:   spec.Name,
		Coin:   (p.Coin - spec.Price).String(),
		Delta:  "-" + spec.Price.String(),
	}, nil
}
// Sell 卖掉一辆空闲车辆，按 85% 回收。
func (s *Service) Sell(ctx context.Context, playerID, vehicleID int64) (TradeResult, error) {
	var out TradeResult
	err := s.store.InTx(ctx, func(tx *sql.Tx) error {
```

```
            v, err := s.store.VehicleIn(ctx, tx, vehicleID)
            if err != nil {
                return ErrBadInput
            }
            if v.OwnerID != playerID {
                return ErrNotYours
            }
            if v.State != gameplay.VehicleIdle {
                return ErrOccupied
            }
            spec, ok := gameplay.VehicleAt(v.Level)
            if !ok {
                return ErrBadInput
            }
            refund := spec.TradeIn(gameplay.ResalePermille)
            if err := s.store.RemoveVehicle(ctx, tx, playerID, vehicleID); err != nil {
                return ErrOccupied
            }
            if err := s.store.AddCoin(ctx, tx, playerID, refund); err != nil {
                return err
            }
            p, err := s.store.PlayerIn(ctx, tx, playerID)
            if err != nil {
                return mapRepoErr(err)
            }
            out = TradeResult{
                Action: "sell",
                Level:    v.Level,
                Name:     spec.Name,
                Coin:     p.Coin.String(),
                Delta:    "+" + refund.String(),
            }
            return nil
        })
        if err != nil {
            return TradeResult{}, err
        }
        return out, nil
}
// CatalogItem 是图鉴里的一行。
type CatalogItem struct {
        Level      int       `json:"level"`
        Name       string `json:"name"`
        Price      string `json:"price"`
        TradeIn    string `json:"trade_in"`
        Hourly     string `json:"hourly"`
        FullPay    string `json:"full_pay"`
        Unlocked   bool      `json:"unlocked"`
```

```
	Affordable bool   `json:"affordable"`
}
// CatalogView 返回图鉴列表，附带「是否解锁」「是否买得起」两个即时判断。
func (s *Service) CatalogView(ctx context.Context, playerID int64) ([]CatalogItem, error) {
	p, err := s.store.PlayerByID(ctx, playerID)
	if err != nil {
		return nil, mapRepoErr(err)
	}
	list := gameplay.Catalog()
	limit := gameplay.UnlockMaxLevel(p.Level)
	out := make([]CatalogItem, 0, len(list))
	for _, v := range list {
}
// ==== 源文件: internal/api/h_vehicle.go ====
package api
import (
	"net/http"
)
// buyReq 是买车请求体。
type buyReq struct {
	Level int `json:"level"`
}
// parkReq 是停车请求体，TargetUID 为空表示一键停车。
type parkReq struct {
	TargetUID string `json:"target_uid"`
}
// vehicles 返回车库列表。
func (a *API) vehicles(w http.ResponseWriter, r *http.Request) {
	self, isIn := currentPlayer(r)
	if !isIn {
		replyErr(w, errNoSession)
		return
	}
	list, err := a.svc.Garage(r.Context(), self.ID)
	if err != nil {
		replyErr(w, err)
		return
	}
	ok(w, map[string]interface{}{"items": list, "count": len(list)})
}
// buy 购买车辆。
func (a *API) buy(w http.ResponseWriter, r *http.Request) {
	self, isIn := currentPlayer(r)
	if !isIn {
		replyErr(w, errNoSession)
		return
	}
	var req buyReq
```

```
	if err := decode(r, &req); err != nil {
		replyErr(w, err)
		return
	}
	result, err := a.svc.Buy(r.Context(), self.ID, req.Level)
	if err != nil {
		replyErr(w, err)
		return
	}
	ok(w, result)
}
// sell 卖车。
func (a *API) sell(w http.ResponseWriter, r *http.Request) {
	self, isIn := currentPlayer(r)
	if !isIn {
		replyErr(w, errNoSession)
		return
	}
	id := pathID(r, "id")
	if id == 0 {
		replyErr(w, errBadPath)
		return
	}
	result, err := a.svc.Sell(r.Context(), self.ID, id)
	if err != nil {
		replyErr(w, err)
		return
	}
	ok(w, result)
}
// park 把车停出去，带目标短号时走精准停车。
func (a *API) park(w http.ResponseWriter, r *http.Request) {
	self, isIn := currentPlayer(r)
	if !isIn {
		replyErr(w, errNoSession)
		return
	}
	id := pathID(r, "id")
	if id == 0 {
		replyErr(w, errBadPath)
		return
	}
	var req parkReq
	// 一键停车允许空体，只有精准停车才需要 target_uid。
	if r.ContentLength > 0 {
		if err := decode(r, &req); err != nil {
			replyErr(w, err)
			return
```

```
            }
        }
        result, err := a.svc.Park(r.Context(), self.ID, id, req.TargetUID)
        if err != nil {
            replyErr(w, err)
            return
        }
        ok(w, result)
}
// recall 召回车辆。
func (a *API) recall(w http.ResponseWriter, r *http.Request) {
        self, isIn := currentPlayer(r)
        if !isIn {
            replyErr(w, errNoSession)
            return
        }
        id := pathID(r, "id")
        if id == 0 {
            replyErr(w, errBadPath)
            return
        }
        result, err := a.svc.Recall(r.Context(), self.ID, id)
        if err != nil {
            replyErr(w, err)
            return
        }
        ok(w, result)
}
// cardSettle 用幸运卡秒结。
func (a *API) cardSettle(w http.ResponseWriter, r *http.Request) {
        self, isIn := currentPlayer(r)
        if !isIn {
            replyErr(w, errNoSession)
            return
        }
        id := pathID(r, "id")
        if id == 0 {
            replyErr(w, errBadPath)
            return
        }
        result, err := a.svc.CardSettle(r.Context(), self.ID, id)
        if err != nil {
            replyErr(w, err)
            return
        }
        ok(w, result)
}
// catalog 返回车辆图鉴。
```

```
func (a *API) catalog(w http.ResponseWriter, r *http.Request) {
	self, isIn := currentPlayer(r)
	if !isIn {
		replyErr(w, errNoSession)
		return
	}
	list, err := a.svc.CatalogView(r.Context(), self.ID)
	if err != nil {
		replyErr(w, err)
		return
	}
	ok(w, map[string]interface{}{"items": list, "count": len(list)})
}
// ==== 源文件: internal/api/h_lot.go ====
package api
import (
	"net/http"
)
// lots 返回我的车位与停在上面的车。
func (a *API) lots(w http.ResponseWriter, r *http.Request) {
	self, isIn := currentPlayer(r)
	if !isIn {
		replyErr(w, errNoSession)
		return
	}
	list, err := a.svc.Lots(r.Context(), self.ID)
	if err != nil {
		replyErr(w, err)
		return
	}
	free := 0
	for _, lot := range list {
		if lot.Occupant == nil {
			free++
		}
	}
	ok(w, map[string]interface{}{
		"items": list,
		"free":  free,
		"total": len(list),
	})
}
// ticket 给停在我家车位上的车贴条。
// 路径参数是车辆 ID，与「我的车位」列表里每辆外来车携带的 vehicle_id 对应。
func (a *API) ticket(w http.ResponseWriter, r *http.Request) {
	self, isIn := currentPlayer(r)
	if !isIn {
		replyErr(w, errNoSession)
```

```
            return
        }
        vehicleID := pathID(r, "id")
        if vehicleID == 0 {
            replyErr(w, errBadPath)
            return
        }
        result, err := a.svc.Ticket(r.Context(), self.ID, vehicleID)
        if err != nil {
            replyErr(w, err)
            return
        }
        ok(w, result)
}
// ==== 源文件: internal/api/h_social.go ====
package api
import (
        "net/http"
        "strconv"
        "tstc/internal/repo"
)
// markReq 是标记关系的请求体。
type markReq struct {
        TargetUID string `json:"target_uid"`
        Kind      string `json:"kind"`
}
// bonds 返回好友或仇人列表，kind 通过查询串传入。
func (a *API) bonds(w http.ResponseWriter, r *http.Request) {
        self, isIn := currentPlayer(r)
        if !isIn {
            replyErr(w, errNoSession)
            return
        }
        kind := r.URL.Query().Get("kind")
        if kind == "" {
            kind = repo.BondFriend
        }
        list, err := a.svc.Bonds(r.Context(), self.ID, kind)
        if err != nil {
            replyErr(w, err)
            return
        }
        ok(w, map[string]interface{}{"items": list, "count": len(list), "kind": kind})
}
// mark 把某人标成好友或仇人。
func (a *API) mark(w http.ResponseWriter, r *http.Request) {
        self, isIn := currentPlayer(r)
        if !isIn {
```

```
            replyErr(w, errNoSession)
            return
        }
        var req markReq
        if err := decode(r, &req); err != nil {
            replyErr(w, err)
            return
        }
        if err := a.svc.Mark(r.Context(), self.ID, req.TargetUID, req.Kind); err != nil {
            replyErr(w, err)
            return
        }
        ok(w, map[string]string{"target_uid": req.TargetUID, "kind": req.Kind})
}
// feed 返回访客动态，hours 缺省为 24 小时。
func (a *API) feed(w http.ResponseWriter, r *http.Request) {
        self, isIn := currentPlayer(r)
        if !isIn {
            replyErr(w, errNoSession)
            return
        }
        hours := 24
        if raw := r.URL.Query().Get("hours"); raw != "" {
            if n, err := strconv.Atoi(raw); err == nil {
                hours = n
            }
        }
        list, err := a.svc.Feed(r.Context(), self.ID, hours)
        if err != nil {
            replyErr(w, err)
            return
        }
        ok(w, map[string]interface{}{"items": list, "count": len(list), "hours": hours})
}
// recommend 返回推荐玩家列表。
func (a *API) recommend(w http.ResponseWriter, r *http.Request) {
        self, isIn := currentPlayer(r)
        if !isIn {
            replyErr(w, errNoSession)
            return
        }
        list, err := a.svc.Recommend(r.Context(), self.ID)
        if err != nil {
            replyErr(w, err)
            return
        }
        ok(w, map[string]interface{}{"items": list, "count": len(list)})
}
```

```
// ==== 源文件: web/index.html ====
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
<meta name="theme-color" content="#101822">
<title>偷偷停个车</title>
<link rel="stylesheet" href="assets/app.css">
</head>
<body>
<header class="topbar">
  <div class="who">
    <div class="avatar" id="avatarBadge">车</div>
    <div class="who-text">
      <div class="nick" id="nickText">未登录</div>
      <div class="uid" id="uidText">—</div>
    </div>
  </div>
  <div class="wallet">
    <div class="chip coin">
      <span class="chip-key">金币</span>
      <strong id="coinText">0</strong>
    </div>
    <div class="chip card">
      <span class="chip-key">幸运卡</span>
      <strong id="cardText">0</strong>
      <button class="chip-btn" id="claimCardBtn" type="button">获取</button>
    </div>
  </div>
</header>
<main class="stage">
  <section class="view" id="view-home" data-view="home">
    <div class="panel sign">
      <div class="panel-head">
        <h2>每日签到</h2>
        <span class="tag" id="signState">今天还没签</span>
      </div>
      <div class="sign-row">
        <button class="btn ghost" id="signPlainBtn" type="button">
          <span class="btn-main">普通签到</span>
          <span class="btn-sub">+5,000 金币</span>
        </button>
        <button class="btn warm" id="signCardBtn" type="button">
          <span class="btn-main">幸运卡签到</span>
          <span class="btn-sub">消耗 1 张 · +50,000 金币</span>
        </button>
      </div>
```

```
      <div class="progress-row">
        <div class="meter"><i id="expMeter"></i></div>
        <span class="hint" id="expText">经验 0 / 0</span>
      </div>
    </div>
    <div class="panel">
      <div class="panel-head">
        <h2>我的车</h2>
        <span class="tag" id="garageCount">0 / 0</span>
      </div>
      <div class="mini-grid" id="homeVehicles"></div>
    </div>
    <div class="panel">
      <div class="panel-head">
        <h2>我的车位</h2>
        <span class="tag" id="lotCount">0 / 0</span>
      </div>
      <div class="mini-grid" id="homeLots"></div>
    </div>
    <div class="panel">
      <div class="panel-head">
        <h2>最近动态</h2>
        <span class="tag">24 小时</span>
      </div>
      <ul class="feed" id="homeFeed"></ul>
    </div>
  </section>
  <section class="view" id="view-garage" data-view="garage" hidden>
    <div class="section-title">
      <h2>车库</h2>
      <p class="hint">空闲的车可以一键派出，也可以选人精准停过去</p>
    </div>
    <div class="card-list" id="garageList"></div>
  </section>
  <section class="view" id="view-lot" data-view="lot" hidden>
    <div class="section-title">
      <h2>我的车位</h2>
      <p class="hint">停在你家的车超过两小时就能贴条，拿走两成收益</p>
    </div>
    <div class="cell-grid" id="lotGrid"></div>
    <div class="card-list" id="lotOccupants"></div>
  </section>
  <section class="view" id="view-contacts" data-view="contacts" hidden>
    <div class="search-box">
      <input id="searchInput" type="text" maxlength="8" placeholder="输入五位玩家号，例如 A8K3M">
      <button class="btn solid" id="searchBtn" type="button">找人</button>
    </div>
    <div class="card-list" id="searchCard"></div>
```

```
      <div class="tabs" id="contactTabs">
        <button class="tab on" data-tab="friend" type="button">好友</button>
        <button class="tab" data-tab="foe" type="button">仇人</button>
        <button class="tab" data-tab="feed" type="button">动态</button>
        <button class="tab" data-tab="near" type="button">推荐</button>
      </div>
      <div class="card-list" id="contactList"></div>
    </section>
    <section class="view" id="view-catalog" data-view="catalog" hidden>
      <div class="section-title">
        <h2>车辆图鉴</h2>
        <p class="hint" id="catalogHint">每升一级解锁一款新车</p>
      </div>
      <div class="card-list" id="catalogList"></div>
    </section>
  </main>
  <nav class="navbar">
    <button class="nav on" data-goto="home" type="button"><span>首页</span></button>
    <button class="nav" data-goto="garage" type="button"><span>车库</span></button>
    <button class="nav" data-goto="lot" type="button"><span>车位</span></button>
    <button class="nav" data-goto="contacts" type="button"><span>通讯录</span></button>
    <button class="nav" data-goto="catalog" type="button"><span>图鉴</span></button>
  </nav>
  <div class="mask" id="modalMask" hidden>
    <div class="modal" role="dialog" aria-modal="true">
      <h3 id="modalTitle">详情</h3>
      <div class="modal-body" id="modalBody"></div>
      <div class="modal-foot" id="modalFoot"></div>
    </div>
  </div>
  <div class="toast" id="toast" hidden></div>
  <script src="assets/core/fmt.js"></script>
  <script src="assets/core/api.js"></script>
  <script src="assets/core/store.js"></script>
  <script src="assets/views/home.js"></script>
  <script src="assets/views/garage.js"></script>
  <script src="assets/views/lot.js"></script>
  <script src="assets/views/contacts.js"></script>
  <script src="assets/views/catalog.js"></script>
  <script src="assets/app.js"></script>
  </body>
  </html>
  // ==== 源文件: web/assets/core/fmt.js ====
  /* 文本与数值格式化：所有展示口径集中在这里，避免各视图各写一套 */
  (function (global) {
    'use strict';
    // 千分位：1234567 → 1,234,567
    function num(value) {
```

```
    var n = Number(value || 0);
    if (!isFinite(n)) {
        return '0';
    }
    var text = String(Math.trunc(Math.abs(n)));
    var out = text.replace(/\B(?=(\d{3})+(?!\d))/g, ',');
    return n < 0 ? '-' + out : out;
}
// 分钟数转成人话：95 → 1 小时 35 分
function minutes(value) {
    var m = Math.max(0, Math.trunc(Number(value || 0)));
    if (m < 60) {
        return m + ' 分钟';
    }
    var h = Math.floor(m / 60);
    var rest = m % 60;
    return rest === 0 ? h + ' 小时' : h + ' 小时 ' + rest + ' 分';
}
// 阶段中文名，与后端 stage.label 保持一致，作为兜底
var STAGE = {
    lock: '绝对锁定期',
    escape: '逃生观察期',
    hunt: '争夺期',
    mature: '期满'
};
function stage(code) {
    return STAGE[code] || '未知阶段';
}
// HTML 转义：所有插入 innerHTML 的动态文本都必须先过这里
function esc(text) {
    return String(text === undefined || text === null ? '' : text)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#39;');
}
// 经验进度条百分比
function ratio(part, total) {
    if (!total || total <= 0) {
        return 0;
    }
    var p = (Number(part) / Number(total)) * 100;
    if (p < 0) {
        return 0;
    }
    if (p > 100) {
        return 100;
```

```
    }
    return Math.round(p);
  }
  global.Fmt = {
    num: num,
    minutes: minutes,
    stage: stage,
    esc: esc,
    ratio: ratio
  };
})(window);
// ==== 源文件: web/assets/core/api.js ====
/* 接口层：统一拼地址、带身份、拆信封。
   服务端返回 { ok, data, error }，这里把 error 变成抛出异常，
   让调用方只写 try/catch，不必到处判断 ok 字段 */
(function (global) {
  'use strict';
  var prefix = '/api/v1';
  var playerUID = '';
  function setUid(uid) {
    playerUID = uid || '';
  }
  function getUid() {
    return playerUID;
  }
  // 错误对象带上 code，界面可据此做分支（例如 no_card 引导去领卡）
  function ApiError(code, text, status) {
    this.name = 'ApiError';
    this.code = code || 'unknown';
    this.message = text || '请求失败';
    this.status = status || 0;
  }
  ApiError.prototype = Object.create(Error.prototype);
  function request(method, path, body) {
    var headers = { 'Accept': 'application/json' };
    if (body !== undefined) {
      headers['Content-Type'] = 'application/json';
    }
    if (playerUID) {
      headers['X-Player-UID'] = playerUID;
    }
    return fetch(prefix + path, {
      method: method,
      headers: headers,
      body: body === undefined ? undefined : JSON.stringify(body)
    }).then(function (res) {
      return res.json().catch(function () {
        throw new ApiError('bad_json', '服务返回的内容无法解析', res.status);
```

```
        }).then(function (payload) {
            if (!payload) {
                throw new ApiError('empty', '服务没有返回内容', res.status);
            }
            if (payload.ok) {
                return payload.data;
            }
            var err = payload.error || {};
            throw new ApiError(err.code, err.text, res.status);
        });
    }, function () {
        throw new ApiError('offline', '网络不太顺畅，稍后再试');
    });
}
function get(path) {
    return request('GET', path);
}
function post(path, body) {
    return request('POST', path, body === undefined ? {} : body);
}
global.Api = {
    setUid: setUid,
    getUid: getUid,
    ApiError: ApiError,
    register: function (nick) {
        return post('/register', { nick: nick || '' });
    },
    me: function () {
        return get('/me');
    },
    rename: function (nick) {
        return request('PATCH', '/me', { nick: nick });
    },
    sign: function (way) {
        return post('/me/sign', { way: way });
    },
    claimCard: function () {
        return post('/me/cards');
    },
    vehicles: function () {
        return get('/vehicles');
    },
    buy: function (level) {
        return post('/vehicles', { level: level });
    },
    sell: function (id) {
        return post('/vehicles/' + id + '/sell');
    },
```

```
    park: function (id, targetUID) {
      return post('/vehicles/' + id + '/park', { target_uid: targetUID || '' });
    },
    recall: function (id) {
      return post('/vehicles/' + id + '/recall');
    },
    cardSettle: function (id) {
      return post('/vehicles/' + id + '/card');
    },
    lots: function () {
      return get('/lots');
    },
    ticket: function (vehicleId) {
      return post('/lots/' + vehicleId + '/ticket');
    },
    bonds: function (kind) {
      return get('/bonds?kind=' + encodeURIComponent(kind));
    },
    mark: function (targetUID, kind) {
      return post('/bonds', { target_uid: targetUID, kind: kind });
    },
    feed: function (hours) {
      return get('/feed?hours=' + (hours || 24));
    },
    recommend: function () {
      return get('/recommend');
    },
    search: function (uid) {
      return get('/players/' + encodeURIComponent(uid));
    },
    catalog: function () {
      return get('/catalog');
    }
  };
})(window);
// ==== 源文件: web/assets/core/store.js ====
/* 前端状态中心：一份数据 + 一组渲染回调。
   任何一次接口返回都只改 state，然后调用 refresh() 让各视图自己重画，
   视图之间不互相调用，避免数据流变成一团乱麻 */
(function (global) {
  'use strict';
  var state = {
    me: null,
    vehicles: [],
    lots: [],
    feed: [],
    bonds: [],
    recommend: [],
```

```
    catalog: [],
    bondKind: 'friend'
};
var painters = [];
var toastTimer = 0;
function onPaint(fn) {
    painters.push(fn);
}
function refresh() {
    for (var i = 0; i < painters.length; i++) {
        try {
            painters[i](state);
        } catch (err) {
            // 单个视图渲染失败不应该拖垮整页
            if (global.console) {
                console.error('视图渲染出错', err);
            }
        }
    }
}
function patch(next) {
    for (var key in next) {
        if (Object.prototype.hasOwnProperty.call(next, key)) {
            state[key] = next[key];
        }
    }
}
function get(key) {
    return state[key];
}
// 轻提示：默认 2 秒后自动消失
function toast(text, bad) {
    var box = document.getElementById('toast');
    if (!box) {
        return;
    }
    box.textContent = text;
    box.className = bad ? 'toast bad' : 'toast';
    box.hidden = false;
    if (toastTimer) {
        clearTimeout(toastTimer);
    }
    toastTimer = setTimeout(function () {
        box.hidden = true;
        toastTimer = 0;
    }, bad ? 3200 : 2000);
}
// 弹窗：rows 是 [标签, 值] 的数组，actions 是 [文案, 回调, 是否主按钮]
```

```
function modal(title, rows, actions) {
  var mask = document.getElementById('modalMask');
  var head = document.getElementById('modalTitle');
  var body = document.getElementById('modalBody');
  var foot = document.getElementById('modalFoot');
  if (!mask || !head || !body || !foot) {
    return;
  }
  head.textContent = title;
  body.innerHTML = rows.map(function (pair) {
    return '<div class="kv"><span>' + global.Fmt.esc(pair[0]) + '</span><b>' +
      global.Fmt.esc(pair[1]) + '</b></div>';
  }).join('');
  foot.innerHTML = '';
  (actions || []).forEach(function (action) {
    var btn = document.createElement('button');
    btn.type = 'button';
    btn.className = 'btn ' + (action[2] ? 'solid' : 'ghost');
    btn.textContent = action[0];
    btn.addEventListener('click', function () {
      closeModal();
      if (typeof action[1] === 'function') {
        action[1]();
      }
    });
    foot.appendChild(btn);
  });
  mask.hidden = false;
}
function closeModal() {
  var mask = document.getElementById('modalMask');
  if (mask) {
    mask.hidden = true;
  }
}
global.Store = {
  onPaint: onPaint,
  refresh: refresh,
  patch: patch,
  get: get,
  toast: toast,
  modal: modal,
  closeModal: closeModal
};
// 点击遮罩关闭弹窗，点内容区不关闭
document.addEventListener('click', function (ev) {
  if (ev.target && ev.target.id === 'modalMask') {
    closeModal();
```

```
    }
  });
})(window);
// ==== 源文件: web/assets/views/home.js ====
/* 首页视图：签到、经验进度、车辆与车位概览、访客动态 */
(function (global) {
  'use strict';
  var Fmt = global.Fmt;
  var Store = global.Store;
  function paintSign(state) {
    var me = state.me;
    if (!me) {
      return;
    }
    var stateText = document.getElementById('signState');
    var plain = document.getElementById('signPlainBtn');
    var card = document.getElementById('signCardBtn');
    if (!stateText || !plain || !card) {
      return;
    }
    stateText.textContent = me.signed_today ? '今天已签到' : '今天还没签';
    plain.disabled = !!me.signed_today;
    card.disabled = !!me.signed_today || me.card < 1;
    card.querySelector('.btn-sub').textContent = me.card < 1
      ? '幸运卡不足，先点右上角获取'
      : '消耗 1 张 · +50,000 金币';
  }
  function paintExp(state) {
    var me = state.me;
    if (!me) {
      return;
    }
    var meter = document.getElementById('expMeter');
    var text = document.getElementById('expText');
    if (!meter || !text) {
      return;
    }
    var need = me.exp_need || 0;
    meter.style.width = need > 0 ? Fmt.ratio(me.exp, need) + '%' : '100%';
    text.textContent = need > 0
      ? '经验 ' + Fmt.num(me.exp) + ' / ' + Fmt.num(need)
      : '已达最高等级';
  }
  // 首页只放概览：车型名 + 状态，点一下跳到车库看细节
  function paintVehicles(state) {
    var box = document.getElementById('homeVehicles');
    var count = document.getElementById('garageCount');
    if (!box || !count) {
```

```
    return;
  }
  if (state.me) {
    count.textContent = state.me.vehicle_count + ' / ' + state.me.vehicle_cap;
  }
  if (!state.vehicles.length) {
    box.innerHTML = '<div class="empty">还没有车，去图鉴买一辆</div>';
    return;
  }
  box.innerHTML = state.vehicles.map(function (v) {
    var cls = v.state === 'parked' ? 'busy' : 'free';
    var tip = v.state === 'parked'
      ? '停在 ' + Fmt.esc(v.parked.host_nick) + ' 的车位'
      : '空闲 · ' + Fmt.num(v.hourly) + ' / 小时';
    return '<div class="mini ' + cls + '"><b>' + Fmt.esc(v.name) + '</b><span>' +
      Fmt.esc(tip) + '</span></div>';
  }).join('');
}
function paintLots(state) {
  var box = document.getElementById('homeLots');
  var count = document.getElementById('lotCount');
  if (!box || !count) {
    return;
  }
  var free = state.lots.filter(function (l) {
    return !l.occupant;
  }).length;
  count.textContent = free + ' 空 / 共 ' + state.lots.length;
  if (!state.lots.length) {
    box.innerHTML = '<div class="empty">车位还没发放，升级后自动增加</div>';
    return;
  }
  box.innerHTML = state.lots.map(function (lot) {
    if (!lot.occupant) {
      return '<div class="mini free"><b>车位 ' + lot.seq + '</b><span>空着</span></div>';
    }
    return '<div class="mini busy"><b>' + Fmt.esc(lot.occupant.vehicle_name) +
      '</b><span>' + Fmt.esc(lot.occupant.stage.label) + '</span></div>';
  }).join('');
}
function paintFeed(state) {
  var box = document.getElementById('homeFeed');
  if (!box) {
    return;
  }
  if (!state.feed.length) {
    box.innerHTML = '<li class="ago">最近还没有人来过</li>';
    return;
```

```
    }
    box.innerHTML = state.feed.slice(0, 5).map(function (item) {
      return '<li><span>' + Fmt.esc(item.kind_text) + '</span><span class="ago">' +
        Fmt.esc(item.ago) + '</span></li>';
    }).join('');
  }
  Store.onPaint(function (state) {
    paintSign(state);
    paintExp(state);
    paintVehicles(state);
    paintLots(state);
    paintFeed(state);
  });
})(window);
// ==== 源文件: web/assets/views/garage.js ====
/* 车库视图：每辆车一张卡，按状态给出可用操作 */
(function (global) {
  'use strict';
  var Fmt = global.Fmt;
  var Store = global.Store;
  var Api = global.Api;
  // 阶段进度条：四段，点亮到当前阶段
  function stageBar(code) {
    var order = ['lock', 'escape', 'hunt', 'mature'];
    var at = order.indexOf(code);
    var parts = order.map(function (name, i) {
      return '<i class="' + (i <= at ? 'on' : '') + '"></i>';
    });
    return '<div class="stage-bar">' + parts.join('') + '</div>';
  }
  function parkedInfo(v) {
    var p = v.parked;
    return '' +
      '<div class="vrow">停在 <em>' + Fmt.esc(p.host_nick) + '</em> （Lv.' + p.host_level +
      '） · 倍率 <em>' + Fmt.esc(p.rate) + '</em></div>' +
      '<div class="vrow">' + stageBar(p.stage.code) + '</div>' +
      '<div class="vrow">已停 <em>' + Fmt.minutes(p.stage.minutes) + '</em> · 累计 <em>' +
      Fmt.num(p.accrued) + '</em> 金币</div>' +
      '<div class="vrow">' + Fmt.esc(p.stage.label) + ' · ' + Fmt.esc(p.stage.hint) + '</div>';
  }
  function idleInfo(v) {
    return '' +
      '<div class="vrow">基础产出 <em>' + Fmt.num(v.hourly) + '</em> / 小时 · 卖价 <em>' +
      Fmt.num(v.trade_in) + '</em></div>';
  }
  function actions(v) {
    var list = [];
    if (v.state === 'parked') {
```

```
        list.push(btn('recall', v.id, '召回', v.parked.can_recall, false));
        list.push(btn('card', v.id, '幸运卡秒结', v.parked.can_card, true));
    } else {
        list.push(btn('park', v.id, '一键停车', true, true));
        list.push(btn('parkTo', v.id, '选人停车', true, false));
        list.push(btn('sell', v.id, '卖掉', true, false));
    }
    return list.join('');
}
function btn(action, id, text, enabled, warm) {
    return '<button type="button" class="btn small ' + (warm ? 'warm' : 'ghost') + '" ' +
        (enabled ? '' : 'disabled ') + 'data-act="' + action + '" data-id="' + id + '">' +
        text + '</button>';
}
function card(v) {
    var parked = v.state === 'parked';
    var stateText = parked ? '停车中' : '空闲';
    return '' +
        '<article class="vcard ' + (parked ? 'parked' : '') + '">' +
        '  <div class="vcard-top">' +
        '    <div>' +
        '      <div class="vname">' + Fmt.esc(v.name) + '</div>' +
        '      <div class="vmeta">Lv.' + v.level + ' · 购入价 ' + Fmt.num(v.price) + '</div>' +
        '    </div>' +
        '    <span class="vstate ' + (parked ? 'on' : '') + '">' + stateText + '</span>' +
        '  </div>' +
        (parked ? parkedInfo(v) : idleInfo(v)) +
        '  <div class="vacts">' + actions(v) + '</div>' +
        '</article>';
}
function paint(state) {
    var box = document.getElementById('garageList');
    if (!box) {
        return;
    }
    if (!state.vehicles.length) {
        box.innerHTML = '<div class="empty">车库里空空如也，去图鉴挑一辆</div>';
        return;
    }
    box.innerHTML = state.vehicles.map(card).join('');
}
// 统一处理车库里的按钮点击
function onAct(ev) {
    var target = ev.target.closest ? ev.target.closest('button[data-act]') : null;
    if (!target) {
        return;
    }
    var id = Number(target.getAttribute('data-id'));
```

```
    var act = target.getAttribute('data-act');
    if (act === 'park') {
      busy(Api.park(id, ''), '已经停出去了');
    } else if (act === 'recall') {
      busy(Api.recall(id), '召回成功');
    } else if (act === 'card') {
      busy(Api.cardSettle(id), '幸运卡已结算');
    } else if (act === 'sell') {
      busy(Api.sell(id), '已卖掉');
    } else if (act === 'parkTo') {
      askTarget(id);
    }
}
// 选人停车：让玩家输入对方短号，停到指定人的空位上
function askTarget(vehicleId) {
    var uid = global.prompt ? global.prompt('输入对方五位玩家号，例如 A8K3M') : '';
    if (!uid) {
      return;
    }
    busy(Api.park(vehicleId, uid.trim().toUpperCase()), '已经停到对方车位');
}
// 所有写操作走同一条路径：调接口 → 提示 → 刷新全部视图
function busy(promise, doneText) {
    promise.then(function (data) {
      var rows = [];
      if (data && data.kind_label) {
        rows.push(['结算方式', data.kind_label]);
        rows.push(['停了几分钟', Fmt.minutes(data.minutes)]);
        rows.push(['毛收益', Fmt.num(data.gross)]);
        rows.push(['你拿到', Fmt.num(data.parker_pay)]);
        if (Number(String(data.host_pay).replace(/,/g, '')) > 0) {
          rows.push(['车位主人拿到', Fmt.num(data.host_pay)]);
        }
        rows.push(['获得经验', data.exp]);
        Store.modal('结算完成', rows, [['知道了', null, true]]);
      } else if (data && data.host_nick) {
        rows.push(['停到', data.host_nick + '（Lv.' + data.host_level + '）']);
        rows.push(['车位号', data.cell_seq]);
        rows.push(['倍率', data.rate]);
        rows.push(['满额收入', Fmt.num(data.full_pay)]);
        Store.modal('停车成功', rows, [['知道了', null, true]]);
      } else {
        Store.toast(doneText);
      }
      return global.App.reload();
    }).catch(function (err) {
      Store.toast(err.message, true);
    });
```

```
  }
  document.addEventListener('click', function (ev) {
    if (ev.target.closest && ev.target.closest('#garageList')) {
      onAct(ev);
    }
  });
  Store.onPaint(paint);
})(window);
// ==== 源文件: web/assets/views/lot.js ====
/* 车位视图：自己的格子 + 停在上面的外来车（可贴条） */
(function (global) {
  'use strict';
  var Fmt = global.Fmt;
  var Store = global.Store;
  var Api = global.Api;
  function cell(lot) {
    if (lot.occupant) {
      return '<div class="cell taken"><b>' + lot.seq + '</b><span>' +
        Fmt.esc(lot.occupant.vehicle_name) + '</span></div>';
    }
    return '<div class="cell"><b>' + lot.seq + '</b><span>空闲</span></div>';
  }
  function occupantCard(lot) {
    var o = lot.occupant;
    var tip = o.can_ticket
      ? '可以贴条，车主拿 ' + Fmt.num(o.parker_pay_if_ticket) + '，你拿 ' + Fmt.num(o.host_pay_if_ticket)
      : '还要等 ' + Fmt.minutes(o.wait_for) + ' 才能贴条';
    return '' +
      '<article class="vcard parked">' +
      '  <div class="vcard-top">' +
      '    <div>' +
      '      <div class="vname">' + Fmt.esc(o.vehicle_name) + '</div>' +
      '      <div class="vmeta">车主 ' + Fmt.esc(o.parker_nick) + '（Lv.' + o.parker_level +
      ') · 车位 ' + lot.seq + '</div>' +
      '    </div>' +
      '    <span class="vstate on">' + Fmt.esc(o.stage.label) + '</span>' +
      '  </div>' +
      '  <div class="vrow">倍率 <em>' + Fmt.esc(o.rate) + '</em> · 已停 <em>' +
      Fmt.minutes(o.stage.minutes) + '</em> · 累计 <em>' + Fmt.num(o.accrued) + '</em></div>' +
      '  <div class="vrow">' + Fmt.esc(tip) + '</div>' +
      '  <div class="vacts">' +
      '    <button type="button" class="btn small warm" data-ticket="' + o.vehicle_id + '" ' +
      (o.can_ticket ? '' : 'disabled ') + '>贴条</button>' +
      '  </div>' +
      '</article>';
  }
  function paint(state) {
    var grid = document.getElementById('lotGrid');
```

```
    var list = document.getElementById('lotOccupants');
    if (!grid || !list) {
      return;
    }
    if (!state.lots.length) {
      grid.innerHTML = '<div class="empty">还没有车位</div>';
      list.innerHTML = '';
      return;
    }
    grid.innerHTML = state.lots.map(cell).join('');
    var taken = state.lots.filter(function (l) {
      return !!l.occupant;
    });
    if (!taken.length) {
      list.innerHTML = '<div class="empty">现在没有车停在你家，等别人送上门</div>';
      return;
    }
    list.innerHTML = taken.map(occupantCard).join('');
  }
  function onTicket(ev) {
    var btn = ev.target.closest ? ev.target.closest('button[data-ticket]') : null;
    if (!btn) {
      return;
    }
    var vehicleId = Number(btn.getAttribute('data-ticket'));
    Api.ticket(vehicleId).then(function (data) {
      Store.modal('贴条完成', [
        ['停了几分钟', Fmt.minutes(data.minutes)],
        ['车主拿到', Fmt.num(data.parker_pay)],
        ['你拿到', Fmt.num(data.host_pay)],
        ['获得经验', data.exp]
      ], [['知道了', null, true]]);
      return global.App.reload();
    }).catch(function (err) {
      Store.toast(err.message, true);
    });
  }
  document.addEventListener('click', function (ev) {
    if (ev.target.closest && ev.target.closest('#lotOccupants')) {
      onTicket(ev);
    }
  });
  Store.onPaint(paint);
})(window);
// ==== 源文件: web/assets/views/contacts.js ====
/* 通讯录视图：按号找人、好友/仇人/动态/推荐四个页签 */
(function (global) {
  'use strict';
```

```
var Fmt = global.Fmt;
var Store = global.Store;
var Api = global.Api;
// 搜索名片专用的两个标记按钮
function markButtons(uid) {
  var safe = Fmt.esc(uid);
  return '<button type="button" class="btn small ghost" data-mark="friend" ' +
    'data-uid="' + safe + '">加好友</button>' +
    '<button type="button" class="btn small ghost" data-mark="foe" ' +
    'data-uid="' + safe + '">标仇人</button>';
}
function goParkButton(uid) {
  return '<button type="button" class="btn small warm" data-gopark="' +
    Fmt.esc(uid) + '">去停车</button>';
}
// 一行人物卡片：昵称 + 一句副标题 + 右侧按钮，search 模式多两个标记按钮
function personCard(item, mode) {
  var sub = 'Lv.' + item.level + ' · 空位 ' + item.free + ' / ' + item.total +
    ' · 去停能拿 ' + item.rate;
  if (item.heat) {
    sub += ' · 结仇 ' + item.heat + ' 次';
  }
  if (item.note) {
    sub += ' · ' + item.note;
  }
  var acts = (mode === 'search' ? markButtons(item.uid) : '') + goParkButton(item.uid);
  return '' +
    '<div class="person">' +
    '  <div class="person-main">' +
    '    <div class="person-name">' + Fmt.esc(item.nick) + '</div>' +
    '    <div class="person-sub">' + Fmt.esc(sub) + '</div>' +
    '  </div>' +
    '  <div class="person-acts">' + acts + '</div>' +
    '</div>';
}
function searchCard(data) {
  var box = document.getElementById('searchCard');
  if (!box) {
    return;
  }
  if (!data) {
    box.innerHTML = '';
    return;
  }
  var relation = data.relation === 'friend' ? '已是好友'
    : data.relation === 'foe' ? '已标记仇人' : '还没关系';
  box.innerHTML = personCard({
    uid: data.uid,
```

```
        nick: data.nick,
        level: data.level,
        free: data.cell_free,
        total: data.cell_total,
        rate: data.rate_if_park,
        note: relation
    }, 'search');
}
// 动态页签的条目与首页同源，这里展示更完整的字段
function feedRow(item) {
    var who = Fmt.esc(item.actor_nick) + '（Lv.' + item.actor_level + '）';
    return '' +
        '<div class="person">' +
        '  <div class="person-main">' +
        '    <div class="person-name">' + who + '</div>' +
        '    <div class="person-sub">' + Fmt.esc(item.kind_text) + '</div>' +
        '  </div>' +
        '  <span class="hint">' + Fmt.esc(item.ago) + '</span>' +
        '</div>';
}
function paintList(state) {
    var box = document.getElementById('contactList');
    if (!box) {
        return;
    }
    if (state.bondKind === 'feed') {
        box.innerHTML = state.feed.length
            ? state.feed.map(feedRow).join('')
            : '<div class="empty">最近还没有动态</div>';
        return;
    }
    if (state.bondKind === 'near') {
        box.innerHTML = state.recommend.length
            ? state.recommend.map(function (item) {
                return personCard(item, 'near');
            }).join('')
            : '<div class="empty">暂时没有推荐的玩家</div>';
        return;
    }
    if (!state.bonds.length) {
        box.innerHTML = state.bondKind === 'friend'
            ? '<div class="empty">还没有好友，搜个号加一个，去他家薅高倍率</div>'
            : '<div class="empty">还没有仇人，被贴条后会自动记进来</div>';
        return;
    }
    box.innerHTML = state.bonds.map(function (item) {
        return personCard(item, 'list');
    }).join('');
```

```
}
function loadBonds(kind) {
    Store.patch({ bondKind: kind });
    if (kind === 'feed' || kind === 'near') {
        Store.refresh();
        return;
    }
    Api.bonds(kind).then(function (data) {
        Store.patch({ bonds: data.items || [] });
        Store.refresh();
    }).catch(function (err) {
        Store.toast(err.message, true);
    });
}
function bindTabs() {
    var box = document.getElementById('contactTabs');
    if (!box) {
        return;
    }
    box.addEventListener('click', function (ev) {
        var tab = ev.target.closest ? ev.target.closest('.tab') : null;
        if (!tab) {
            return;
        }
        var all = box.querySelectorAll('.tab');
        for (var i = 0; i < all.length; i++) {
            all[i].className = all[i] === tab ? 'tab on' : 'tab';
        }
        loadBonds(tab.getAttribute('data-tab'));
    });
}
function bindSearch() {
    var input = document.getElementById('searchInput');
    var button = document.getElementById('searchBtn');
    if (!input || !button) {
        return;
    }
    button.addEventListener('click', function () {
        var uid = (input.value || '').trim().toUpperCase();
        if (uid.length < 4) {
            Store.toast('请输入完整的玩家号', true);
            return;
        }
        Api.search(uid).then(function (data) {
            searchCard(data);
        }).catch(function (err) {
            searchCard(null);
            Store.toast(err.message, true);
```

```
        });
      });
    }
    // 好友/仇人标记与「去停车」统一走事件委托
    document.addEventListener('click', function (ev) {
      var markBtn = ev.target.closest ? ev.target.closest('button[data-mark]') : null;
      if (markBtn) {
        var uid = markBtn.getAttribute('data-uid');
        var kind = markBtn.getAttribute('data-mark');
        Api.mark(uid, kind).then(function () {
          Store.toast(kind === 'friend' ? '已加为好友' : '已标记为仇人');
          loadBonds(kind);
        }).catch(function (err) {
          Store.toast(err.message, true);
        });
        return;
      }
      var parkBtn = ev.target.closest ? ev.target.closest('button[data-gopark]') : null;
      if (parkBtn) {
        global.App.parkTo(parkBtn.getAttribute('data-gopark'));
      }
    });
    bindTabs();
    bindSearch();
    Store.onPaint(paintList);
})(window);
// ==== 源文件: web/assets/views/catalog.js ====
/* 图鉴视图：1 到 100 级车型一览，可购买 / 已拥有 / 未解锁三种状态 */
(function (global) {
    'use strict';
    var Fmt = global.Fmt;
    var Store = global.Store;
    var Api = global.Api;
    function row(item, state) {
      var owned = state.vehicles.some(function (v) {
        return v.level === item.level;
      });
      var action;
      if (!item.unlocked) {
        action = '<button type="button" class="btn small ghost" disabled>等级不够</button>';
      } else if (!item.affordable) {
        action = '<button type="button" class="btn small ghost" disabled>金币不够</button>';
      } else {
        action = '<button type="button" class="btn small warm" data-buy="' +
          item.level + '">购买</button>';
      }
      return '' +
        '<div class="person">' +
```

```
      '   <div class="person-main">' +
      '     <div class="person-name">Lv.' + item.level + ' ' + Fmt.esc(item.name) + '</div>' +
      '     <div class="person-sub">' +
      '       价格 ' + Fmt.num(item.price) +
      ' · 产出 ' + Fmt.num(item.hourly) + '/时' +
      ' · 满额 ' + Fmt.num(item.full_pay) +
      ' · 卖价 ' + Fmt.num(item.trade_in) +
      (owned ? ' · <em>已拥有</em>' : '') +
      '     </div>' +
      '   </div>' +
      '   <div class="person-acts">' + action + '</div>' +
      '</div>';
}
function paint(state) {
  var box = document.getElementById('catalogList');
  var hint = document.getElementById('catalogHint');
  if (!box) {
    return;
  }
  if (hint && state.me) {
    hint.textContent = '当前等级 Lv.' + state.me.level + '，可购买到 Lv.' + state.me.max_buy_level;
  }
  if (!state.catalog.length) {
    box.innerHTML = '<div class="empty">图鉴加载中</div>';
    return;
  }
  // 只展示当前等级附近的一段，避免一次渲染 100 行
  var limit = state.me ? state.me.max_buy_level + 3 : 10;
  var list = state.catalog.filter(function (item) {
    return item.level <= limit;
  });
  box.innerHTML = list.map(function (item) {
    return row(item, state);
  }).join('');
}
document.addEventListener('click', function (ev) {
  var btn = ev.target.closest ? ev.target.closest('button[data-buy]') : null;
  if (!btn) {
    return;
  }
  var level = Number(btn.getAttribute('data-buy'));
  Api.buy(level).then(function (data) {
    Store.toast('买到 ' + data.name + '，余额 ' + Fmt.num(data.coin));
    return global.App.reload();
  }).catch(function (err) {
    Store.toast(err.message, true);
  });
});
```

```
  Store.onPaint(paint);
})(window);
// ==== 源文件: web/assets/app.js ====
/* 应用入口：恢复身份、拉数据、绑定导航与顶部按钮 */
(function (global) {
  'use strict';
  var Store = global.Store;
  var Api = global.Api;
  var UID_KEY = 'tstc.player.uid';
  var currentView = 'home';
  var ticking = false;
  // 从本地缓存恢复身份，没有就登记一个新号
  function ensureIdentity() {
    var cached = '';
    try {
      cached = global.localStorage.getItem(UID_KEY) || '';
    } catch (err) {
      cached = '';
    }
    if (cached) {
      Api.setUid(cached);
      return Promise.resolve(cached);
    }
    var seed = '车主' + Math.floor(Math.random() * 9000 + 1000);
    return Api.register(seed).then(function (data) {
      var uid = data.player.uid;
      Api.setUid(uid);
      try {
        global.localStorage.setItem(UID_KEY, uid);
      } catch (err) {
        // 隐私模式下写不进缓存也不影响本次会话
      }
      return uid;
    });
  }
  // 顶部资产区：昵称、短号、金币、幸运卡
  function paintTop(state) {
    var me = state.me;
    if (!me) {
      return;
    }
    document.getElementById('nickText').textContent = me.nick + ' · Lv.' + me.level;
    document.getElementById('uidText').textContent = '玩家号 ' + me.uid;
    document.getElementById('coinText').textContent = Fmt.num(me.coin_raw);
    document.getElementById('cardText').textContent = me.card;
    document.getElementById('avatarBadge').textContent = me.nick.slice(0, 1) || '车';
    var claim = document.getElementById('claimCardBtn');
    claim.disabled = me.ad_left <= 0;
```

```
    claim.textContent = me.ad_left > 0 ? '获取' : '今日已满';
}
var Fmt = global.Fmt;
// 一次性拉齐首页需要的四份数据，再统一刷新视图
function reload() {
    return Promise.all([
        Api.me(),
        Api.vehicles(),
        Api.lots(),
        Api.feed(24)
    ]).then(function (all) {
        Store.patch({
            me: all[0],
            vehicles: all[1].items || [],
            lots: all[2].items || [],
            feed: all[3].items || []
        });
        // 通讯录里的好友/仇人按当前页签决定要不要一起刷
        if (currentView === 'contacts') {
            return loadBonds(Store.get('bondKind'));
        }
        return null;
    }).then(function () {
        Store.refresh();
    }).catch(function (err) {
        Store.toast(err.message || '加载失败', true);
    });
}
// 通讯录数据按页签懒加载，避免每次刷新都多打两个接口
function loadBonds(kind) {
    if (kind === 'feed' || kind === 'near') {
        if (kind === 'near') {
            return Api.recommend().then(function (data) {
                Store.patch({ recommend: data.items || [] });
            }).catch(function () {
                Store.patch({ recommend: [] });
            });
        }
        return Promise.resolve(null);
    }
    return Api.bonds(kind).then(function (data) {
        Store.patch({ bonds: data.items || [] });
    }).catch(function () {
        Store.patch({ bonds: [] });
    });
}
```